

Geometric Grammar Systems: Architecting Polygon-Based Syntax for Cross-Domain Generative Composition in Interfaces, Images, and Built Environments

I. Introduction: Architecting Meaning with Polygons - Towards a Generative Geometric Grammar

A. The Proposition: Polygons as Generative Syntax

The rapid advancement of generative artificial intelligence presents profound opportunities and challenges for design disciplines. While models demonstrate remarkable capacity for synthesis, controlling their output with precision and intentionality remains a critical frontier. This report investigates the potential of formalizing fundamental geometric primitives—specifically polygons—into a structured **Polygonal Grammar**. The central hypothesis posits that polygons, ranging from simple triangles to complex n-gons, can function beyond mere descriptive elements in prompts, serving instead as a foundational, cross-domain syntax—a **generative geometric grammar**—for architecting both structure and meaning in computational design workflows. This inquiry moves beyond natural language description towards a system where geometric units operate as modular components within a rule-based framework, influencing generative outcomes across diverse domains such as visual art, interface design, and architectural conception. The context for this exploration is the increasing sophistication of generative models like Stable Diffusion and Midjourney¹ and the corresponding need for prompt engineering techniques that offer structured, symbolic control over the generative process.

B. Rationale and Potential Impact

The development of a shared polygonal grammar offers significant potential benefits. Such a system could enable more predictable, controllable, and transferable generative design processes. By defining explicit rules for geometric composition, designers might achieve greater fidelity between intent and outcome. Furthermore, a common grammar could facilitate the translation of structural and semantic logic across domains, allowing, for instance, the principles of an interface layout to inform the composition of a generated image or the spatial organization of a conceptual architectural plan. This could foster novel forms of computational creativity, enabling explorations that fluidly bridge visual, spatial, and structural thinking. This investigation builds upon the legacy of established concepts such as **Shape Grammars**³ and procedural content generation techniques⁵, but adapts these principles to the specific context of contemporary, prompt-driven generative AI systems, focusing explicitly on the syntactic and semantic role of polygons.

C. Research Objectives and Structure

This report aims to rigorously explore the feasibility and characteristics of a polygonal grammar system. The primary objectives are:

1. To develop a typology of polygonal roles across image generation, interface design, and architecture.
2. To define a formal grammar with explicit syntax rules governing polygon relationships (adjacency, symmetry, scale).
3. To examine the application of recursive polygonal grammars for generating complex and fractal structures.
4. To analyze the use of polygonal logic for generating spatial hierarchy in user interfaces.
5. To identify the limitations, failure modes, and potential biases inherent in such a system.
6. To critically reflect on the underlying assumptions, particularly concerning geometric formalism and cultural

universality.

The report is structured to address these objectives systematically. Section II establishes the theoretical foundations in shape grammars and visual semiotics. Section III defines the proposed polygonal grammar, including its syntax and rules, referencing advanced formalisms like L-Systems and Geometric Graph Grammars. Section IV explores potential cross-domain applications. Section V focuses on recursion and fractal generation. Section VI critically examines limitations, failure modes, and inherent biases. Finally, Section VII synthesizes the findings and outlines future research directions.

II. Theoretical Foundations: Geometry, Grammar, and Semiosis

A. Shape Grammars: A Generative Precedent

The concept of using formal rules to generate geometric forms has a strong precedent in **Shape Grammars**, introduced by George Stiny and James Gips in the early 1970s.³ Shape grammars are production systems specifically designed for generating geometric shapes, typically in 2D or 3D. They provide a mathematical and algorithmic foundation for studying spatial languages.³

A shape grammar consists of a set of **shape rules** and a **generation engine**. Each rule defines how an existing shape (or part of a shape) can be transformed. A rule comprises a Left-Hand Side (LHS), which depicts a shape condition to be matched, and a Right-Hand Side (RHS), which shows how the matched shape should be replaced or augmented.³ Markers are often used to help locate and orient the replacement shape. The generation process typically starts with an initial shape (defined by a start rule), proceeds through the application of transformation rules, and potentially concludes when a termination condition is met.³

A key distinction from Chomsky's linguistic grammars is that shape grammar rules can be applied in parallel—simultaneously to all matching occurrences within the current working shape—a feature shared with L-Systems but not typically allowed in Chomsky grammars.³ Shape grammars notably abolish the distinction between terminal and non-terminal symbols found in traditional formal language theory; the symbols rewritten are the same geometric entities that appear in the final design.⁴

Historically, shape grammars were applied to painting and sculpture³ but found significant traction in architecture and computer-aided architectural design (CAAD) as a formalism for creating novel designs and analyzing existing styles.³ Examples include analyses of Palladian villas, Frank Lloyd Wright's prairie houses⁷, and traditional housing layouts.³

Despite their theoretical elegance and generative power, the practical implementation of shape grammars faces significant challenges, particularly concerning **ambiguity** and **emergence**. A core capability of shape grammars, especially in creative contexts, is the ability to recognize shapes embedded within other shapes in ways not explicitly defined by the initial structure—seeing new figures emerge as rules are applied.⁸ Recreating this dynamic re-interpretation computationally is difficult, as standard CAD systems often rely on fixed topological structures that resist such fluidity.⁹ General implementations that can handle unrestricted rule application and the recognition of emergent, potentially parametric shapes remain an unsolved problem, often requiring NP-hard computations.⁹ Consequently, many practical applications adopt fixed part structures, limiting the grammar's expressive power and potentially making full implementation impossible.⁹ This historical difficulty in computationally realizing the full potential of shape grammars, especially their capacity for dynamic re-structuring and emergence, serves as a crucial consideration for developing a polygonal grammar intended for use with contemporary AI systems, which

might similarly default to more rigid interpretations.

B. Visual Semiotics: How Geometric Forms Convey Meaning

While shape grammars provide a formalism for geometric manipulation, a truly effective design language must also account for meaning. **Semiotics**, the study of signs and symbols and their interpretation, offers a framework for understanding how geometric forms communicate.¹¹ At its core, semiotics examines the process of extracting meaning from sensory perception. In visual terms, it's how we see something (a shape, an image) and process what it is and what it signifies.¹¹ Ferdinand de Saussure's dyadic model posits a relationship between the 'signifier' (the physical form, e.g., a drawn triangle) and the 'signified' (the concept it represents, e.g., 'stability' or 'triangle').¹¹ Charles Sanders Peirce proposed a triadic model involving the sign (representation), the object (what it stands for), and the interpretant (the sense made of the sign).¹²

The application of semiotics is evident in the design of geometric diagrams, such as those found in mathematics textbooks. These diagrams are not merely illustrations but structured visual arguments that use specific visual features to convey meaning about abstract mathematical entities.¹³ For instance, variations in stroke weight can differentiate given information from what needs to be proven, or emphasize important elements ('look here'). Color can group related parts. Specific markings, like small quadrants or dotted lines, explicitly denote properties like perpendicularity or parallelism.¹³ These visual conventions form a system that guides the viewer's interpretation and structures their interaction with the diagram, conveying visual analogues of linguistic moods like declarative ("this is important") or interrogative ("will this help?").¹³

This highlights that geometric forms are not semantically neutral. Their interpretation is guided by learned conventions and context. A polygonal grammar, therefore, cannot operate solely on geometric properties. To function as a cross-domain *language*, it must engage with the semiotic layer. The rules need to consider the meanings associated with polygons, acknowledging that these meanings are often culturally constructed and context-dependent.¹¹ As explored by the theory of **cultural logic**, shared systems of meaning, built upon common attention to accessible semiotic structures (including visual ones), enable individuals within a society to interpret actions and phenomena in predictably convergent ways.¹⁵ If a triangle signifies 'stability' in one cultural or design context¹⁶ but 'tension' or 'warning' in another, a robust polygonal grammar must account for this polysemy. It requires mechanisms beyond pure geometry, potentially incorporating semantic tags, contextual parameters, or rules that explicitly map form to function or meaning within specific domains. Without this semantic grounding, a purely syntactic grammar risks generating geometrically valid but semantically incoherent or inappropriate designs.

C. The Symbolic Lexicon of Polygons

Investigating the specific meanings attributed to polygons reveals a rich tapestry of associations spanning design psychology, cultural symbolism, and sacred geometry.

- **Circles:** Universally recognized, circles often symbolize unity, wholeness, eternity, perfection, and the cyclical nature of life.¹⁶ Having no beginning or end, they represent equilibrium and interconnectedness.¹⁶ In design, they convey community and possibility (e.g., logos like Google Chrome, Target)¹⁶ and are associated with softer, comforting emotions.¹⁶ In sacred contexts, they appear as mandalas, halos, and symbols of the cosmos.¹⁷
- **Triangles:** These are dynamic shapes associated with balance, harmony, and trinities (e.g., mind-body-spirit).¹⁶ They can represent strength and stability, particularly when pointing downwards, but also

change, conflict, or tension.¹⁶ Their orientation heavily influences interpretation. Architecturally, they often signify ascension or hierarchy.¹⁷

- **Squares and Rectangles:** These forms evoke stability, order, reliability, trust, and dependability.¹⁶ They represent the material world, groundedness, and security, often used in architectural foundations and layouts to symbolize structure and order within human civilization (e.g., representing the four cardinal directions).¹⁷
- **Hexagons:** As polygons with multiple sides, hexagons symbolize complexity, balance, and harmony.¹⁶ They are strongly associated with efficiency and structure due to their natural occurrence (honeycombs¹⁹, basalt columns²⁰) and optimal tessellating properties.²¹ This efficiency makes them prevalent in UI grids²¹ and architectural tiling.¹⁹ Symbolically, their six sides are linked to the six days of creation, harmony (via two interlocking triangles representing male/female energy), life potential, and spiritual attainment.¹⁹
- **Other Polygons:** Pentagons (five sides) can represent the five elements, while octagons (eight sides) may symbolize regeneration or infinity.¹⁶ Star polygons carry diverse meanings depending on the number of points and cultural context, often representing unity (flags), authority (sheriff's badges), religious identity (Star of David, Islamic patterns, Star of Bethlehem), or mystical significance (pentagram).²³ Eight-pointed stars are common in Arabic art (Rub el Hizb).²³

Sacred Geometry further imbues polygons with significance, positing that certain geometric forms and proportions (like the Platonic Solids—tetrahedron, cube, octahedron, dodecahedron, icosahedron—and the Golden Ratio, $\phi \approx 1.618$) reflect universal patterns of order and harmony.¹⁷ These principles are believed to be embedded in nature and have been consciously applied in the design of sacred art and architecture across cultures (e.g., Egyptian pyramids, Greek Parthenon, Gothic cathedrals, Hindu mandalas) to evoke balance, beauty, and connection to the divine.¹⁷ Polygons are fundamental building blocks within these systems.

The following table synthesizes these associations, mapping polygons to their roles and meanings across the target domains. This lexicon is crucial for informing a semantically aware polygonal grammar, providing a foundation for prompts that leverage both geometric form and intended meaning.

Table 1: Cross-Domain Polygon Symbolism and Function Lexicon

Polygon	Domain(s)	Symbolic Meaning(s)	Structural Role(s)	Example Snippets
Circle	Image, UI, Arch	Unity, wholeness, infinity, eternity, perfection, cycles, community, comfort, divinity, interconnectedness	Framing device, focal point (center), radial menu base, container, cyclical process indicator	16
Triangle	Image, UI, Arch	Balance, harmony, trinity, strength, stability (esp. downward), dynamism, change, conflict, tension, ascension	Compositional element (rule of thirds), directional cue, hierarchical indicator, structural support (truss), warning	16

Square/Rectangle	Image, UI, Arch	Stability, order, reliability, dependability, groundedness, security, material world, structure	Grid cell, container, frame, foundational element, module, text box	16
Hexagon	Image, UI, Arch	Complexity, balance, harmony, efficiency, structure, community, nature, life potential, creation, harmony	Tessellating grid unit (UI, maps), floor plan module, structural element (honeycomb), decorative motif	16
Pentagon	Image, Arch	Five elements, human form (microcosm), protection (pentagram)	Building block (dodecahedron), decorative motif, framing element	16
Octagon	Image, Arch	Regeneration, infinity, transition, balance	Floor plan shape, decorative element (Islamic art), stop sign affordance	16
Star Polygons	Image, Arch, Design	Unity, authority, religion (Islam, Judaism, Christianity), guidance, military, magic/mysticism	Emblematic symbol (flags, logos), decorative motif (Islamic patterns), structural element (implied)	23

III. Defining the Polygonal Grammar: Syntax, Rules, and Composition

Building upon the theoretical foundations of shape grammars and semiotics, this section outlines the components of a proposed Polygonal Grammar System designed for prompt-based generative workflows.

A. Elements of the Grammar: Polygons as Terminals/Non-Terminals

The fundamental units, or **symbols**, of this grammar are polygons, defined by their number of sides (triangle, square, pentagon, hexagon,..., n-gon). Following the approach of shape grammars⁴, this system may initially abolish the strict distinction between terminal (final) and non-terminal (intermediate, replaceable) symbols. A polygon in a prompt could represent a final geometric element or act as a placeholder to be elaborated by subsequent rules or model interpretation.

Crucially, these symbols are not merely abstract labels but **geometric entities** possessing spatial properties: position (coordinates x,y,z), size (scale factor or dimensions), and orientation (rotation angle).²⁶ These properties are integral to the symbol's definition and manipulation within the grammar. This contrasts with traditional graph grammars where nodes might only represent topology, requiring separate mechanisms for geometric embedding.²⁶

B. Syntactic Rules: Encoding Relationships

The power of the grammar lies in its **syntactic rules**, which define how polygon symbols can relate to one another

spatially and structurally. Key rule types include:

- 1. Adjacency:** These rules specify how polygons can connect or be positioned relative to each other. Constraints might dictate which types of polygons can share an edge, whether edges must align fully or partially, or required distances between centers. Inspiration can be drawn from techniques like the **Polygons-in-Contact (PIC)** method used in Islamic pattern generation, where motifs within adjacent polygons are constructed based on rays emanating from edge midpoints at specific "contact angles" (θ).²⁴ This ensures geometric continuity across boundaries.
- 2. Symmetry:** Rules can enforce or explicitly break symmetry. This includes specifying types of symmetry (e.g., reflectional across an axis, rotational around a point by n degrees) and the scope of application (within a single polygon, between adjacent polygons, across a larger composition). Symmetry is a powerful compositional tool, creating harmony and balance, but can also lead to predictability if overused.²⁹ Islamic geometric patterns demonstrate sophisticated use of rotational and reflectional symmetry, often incorporating rosettes with complex orders (e.g., 7, 9, 11, 13-fold symmetry) within larger symmetric structures.²⁴
- 3. Scaling/Modulation:** These rules govern how changes in a polygon's size impact its function or the surrounding composition. Scaling can signify visual hierarchy (larger elements are more important), emphasis, or a change in functional role.³⁰ Rules might define proportional relationships between scaled elements or trigger changes in adjacent elements. This connects to the concept of **fractal scaling**, where patterns repeat at different magnitudes, a key characteristic of fractal geometry.³¹
- 4. Hierarchy & Grouping:** Rules are needed to define nested structures and composite elements. Syntax like bracket notation `` could group polygons A and B into a single unit that can be manipulated (e.g., repeated, scaled) together.⁵ This allows for building complex assemblies from simpler parts. Geometric hierarchy (e.g., a large polygon containing smaller ones) can be mapped to functional or semantic hierarchy (e.g., a main content area containing sub-modules).³⁰

C. Compositional Templates: Building Logic Blocks

By combining basic polygons and syntactic rules, **compositional templates** or "logic blocks" can be defined. These represent reusable patterns or structural motifs. Examples include:

- HexTileUnit = (Hexagon + Adjacency(EdgeAlign: Full) + Symmetry(Rotational: 60deg))
- RadialMenuCore = (Circle(Center) + Adjacent(Sector(N), RadialAlign: Center))
- RuleOfThirdsScaffold = (Rectangle(Frame) + Point(Intersection: TopLeft) + Point(Intersection: TopRight) + ...)

These templates could encode common layout strategies (grids, radial arrangements, centered compositions²⁹) or specific structural configurations. They form the building blocks for constructing more complex prompts, allowing designers to work at a higher level of abstraction.

The following table proposes a basic syntax for such a grammar, providing a starting point for a mini-language suitable for prompt engineering.

Table 2: Polygonal Grammar Syntax Sheet (Conceptual)

Category	Symbol/Syntax	Description	Example
Elements	Polygon(N)	Defines a regular polygon with N sides (e.g., Polygon(3) for triangle, Polygon(6) for	Polygon(4)

		hexagon).	
	Shape(Name)	References a predefined complex shape or irregular polygon.	Shape(Star5)
	Point(x,y) or Point(ID)	Defines a specific point or references a named point (e.g., center, vertex).	Point(Center)
Properties	.Pos(x,y,z)	Sets absolute or relative position.	Polygon(4).Pos(0,0,0)
	.Scale(s) or .Size(w,h)	Sets uniform scale or specific dimensions.	Polygon(6).Scale(2.0)
	.Rot(angle, axis)	Sets rotation.	Polygon(3).Rot(45, Z)
	.Attr(name=value)	Assigns arbitrary attributes (e.g., color, function).	Polygon(4).Attr(color=blue)
Relationships	Adjacent(ShapeA, ShapeB, Type)	Specifies adjacency constraint (e.g., Edge, Vertex, Distance).	Adjacent(PolyA, PolyB, EdgeAlign)
	Inside(ShapeA, ShapeB)	Specifies containment.	Inside(CircleSmall, CircleBig)
	Symmetry(Type, Ref)	Enforces symmetry (e.g., Reflect(Axis), Rotate(Point, Angle)).	Symmetry(Reflect(Y_Axis))
Operators	+	Combines elements sequentially or additively.	Polygon(4) + Polygon(3)
	``	Groups elements into a composite unit.	[Polygon(4) + Point(Center)]
	*N or *	Repeats the preceding element or group N times or as many times as possible.	[Polygon(6)]*6
	`	or< >`	Represents choice or probability between elements/groups.
Recursion	Rule(LHS -> RHS)	Defines a replacement rule (similar to L-Systems/Shape Grammars).	Rule(A -> F[+A][-A])
	Iterate(RuleSet, N)	Applies a set of rules for N generations.	Iterate(MyRules, 5)

D. Advanced Formalisms: L-Systems and Geometric Graph Grammars (GGG)

To handle more complex generative processes, particularly involving recursion and explicit geometric reasoning, established formalisms can be integrated or adapted.

1. **L-Systems (Lindenmayer Systems):** L-systems are parallel rewriting systems excellent for generating self-similar structures and simulating growth processes.³³ They use an initial string (axiom or premise) and a set of production rules to iteratively rewrite the string. The resulting string is interpreted as drawing commands for a "turtle" graphics system.³³ Key commands include moving forward (F), turning (+, -), pitching (&, ^), rolling (\, /), scaling size (" , _) or angle (;, @), pushing/popping state for branching ([,]), and crucially, defining polygons ({, ., }).³³ The number of iterations (generations) controls the complexity.³³ L-systems provide a robust, well-understood syntax for encoding recursive polygonal generation, directly relevant to the query's interest in fractal prompting.³³
2. **Geometric Graph Grammars (GGG):** GGGs extend traditional graph grammars by explicitly incorporating geometric information (spatial position) into the graph structure itself.²⁶ A GGG production rule typically replaces a node (q) with a subgraph (S), using a **geometric embedding** (B) to specify how the new subgraph connects to the existing graph and how its vertices are positioned, often relative to local coordinate systems and transformations (T_i).²⁶ This allows the grammar to reason directly about spatial relationships and geometric properties. A particularly compelling aspect of GGGs is the potential for **inverse procedural modeling**: algorithms exist that can automatically learn GGG rules by analyzing existing geometric structures (e.g., road networks, potentially design examples) and identifying repeated patterns and connectivity.²⁶ This learning capability is highly significant. While L-systems require manual rule definition, the potential for a GGG-based system to *learn* effective polygonal grammar rules from examples offers a pathway towards more adaptive, data-driven, and potentially less labor-intensive grammar development. This could bridge the gap between predefined rules and the complexities observed in real-world designs.

Integrating concepts from L-Systems (for recursion syntax) and GGGs (for explicit geometry and inverse modeling potential) could significantly enhance the power and applicability of a polygonal grammar system for prompt-based generation.

IV. Cross-Domain Generative Applications

A core proposition of the Polygonal Grammar System is its potential for cross-domain applicability. A well-defined grammar could serve as a shared language for structuring prompts intended for image generation, user interface design, and architectural conception, allowing underlying logic to be preserved while adapting to domain-specific interpretations.

A. Image Generation

In the realm of image synthesis, a polygonal grammar could offer structured control beyond textual descriptions:

- **Compositional Scaffolding:** Polygons can define the underlying structure of an image before detailed rendering. They can delineate major zones, establish lines of force, or position focal points according to compositional principles like the Rule of Thirds, where key elements are placed at the intersections of a 3x3 grid.³⁰
- **Visual Weight and Framing:** The type, size, and placement of grammatical polygons can guide the viewer's attention. A centrally placed polygon naturally draws focus.²⁹ Larger polygons might signify importance, while

specific shapes could frame the primary subject, creating visual hierarchy and directing the narrative of the image.

- **Pattern Generation:** The grammar can generate complex textures, backgrounds, or ornamental details. Tessellations using regular polygons like hexagons²¹ or squares can create repeating surface patterns. Recursive rules can generate intricate fractal designs⁷ or non-periodic patterns inspired by Islamic geometric art, potentially using techniques like Polygons-in-Contact (PIC) or Wheel Construction adapted for image synthesis.²⁴

B. Interface Design (UI/UX)

For user interface and user experience design, polygonal grammar offers methods for generating layout and interaction logic:

- **Spatial Hierarchy & Zoning:** Instead of relying solely on heuristics, interface zones (e.g., navigation, content, input fields, status indicators) could be generated based on geometric relationships defined by the grammar. Nested polygons might represent containment or levels of interaction, while adjacency rules define functional groupings. The geometric properties (size, centrality, shape type) could directly map to the perceived importance or function of UI elements [Query Lens 4].
- **Modular Layouts:** Tessellating polygons provide a natural basis for creating modular and responsive grid systems. While square grids are common, **hexagonal grids** offer distinct advantages relevant to UI and data visualization:
 - *Uniform Adjacency:* The distance between the center of a hexagon and the center of any adjacent hexagon is constant, simplifying neighbor calculations and spatial queries.²¹
 - *Efficiency:* Hexagons have the lowest perimeter-to-area ratio among tessellating polygons, minimizing edge effects and sampling bias in data aggregation.²¹
 - *Geometric Properties:* They better approximate curves and suffer less distortion over large (potentially spherical, like global map data) surfaces.²¹ The H3 indexing system is a practical example leveraging these benefits for global geospatial data.²¹ These properties make hexagonal grids suitable for dashboards, data visualizations, and potentially novel input methods.
- **Navigation Logic:** Polygonal structures can inform navigation design. **Radial menus** (or pie menus) explicitly use circular or polygonal segments arranged around a center point.³⁵ These leverage direction-based selection, which can be very fast for experienced users relying on muscle memory.³⁶ However, usability studies indicate potential trade-offs: while navigation time can be faster than traditional linear menus (by 20-30% in some studies), visual search time might increase.³⁵ Radial menus typically accommodate a limited number of items effectively (around 3-12).³⁶ Polygonal grammar could also define **nested navigation** structures, where geometric layouts visually represent the hierarchy and relationship between different levels or sections of an application.³²

C. Architecture & Built Environment

In architecture, polygonal grammar connects to both historical practices and computational design:

- **Structural Logic:** Polygons can represent fundamental structural elements or systems. Shape grammars originated partly within CAAD³, demonstrating the historical link between rule-based geometry and architectural design. Grammatical rules could define how structural modules connect or how load-bearing forms are configured.
- **Spatial Planning & Floorplan Efficiency:** The grammar can generate floor plans based on polygonal

modules. Hexagons, for example, are noted for their packing efficiency (as seen in honeycombs¹⁹) and could be used to design pavilions or clustered spaces [Query Lens 1]. Rules governing adjacency and circulation could optimize spatial layouts based on geometric principles.

- **Symbolic and Sacred Geometry:** Architectural design has long employed geometry for symbolic purposes. A polygonal grammar could explicitly incorporate principles of sacred geometry, using specific shapes (squares in Hindu temples¹⁸, circles in mandalas¹⁸, star polygons) and proportions (like the Golden Ratio found in the Parthenon and pyramids¹⁷) to imbue designs with cultural or spiritual meaning, aiming for Vitruvian ideals of beauty, durability, and utility.¹⁷

D. Transferable Prompt Structures: The Shared Grammar

The central hypothesis is that a single grammatical prompt structure can be interpreted across these domains, preserving core spatial or logical relationships while yielding domain-specific outputs. Consider a prompt encoding a central hexagon surrounded by six smaller ones: `Template(CenterSurround, Center=Polygon(6).Scale(2), Surround=Polygon(6).Scale(1), Count=6, Adjacency=EdgeAlign)`.

- **Image Generation:** This might produce an image with a large central subject visually framed or supported by six related, smaller elements arranged hexagonally.
- **Interface Design:** This could translate into a UI layout with a main hexagonal content area and six peripheral widgets or menu items in a radial configuration.
- **Architecture:** This might generate a floor plan for a central hexagonal hall connected to six smaller, adjoining rooms or alcoves.

(Visualization) Prompt Grammar Map: This cross-domain translation can be visualized as a map where different syntactic structures (e.g., linear sequence $A+B+C$, nested structure $A[C]$, grid $[\text{Hex}]^*$, recursive pattern $\text{Rule}(X \rightarrow F[+X]X)$) are shown branching into distinct but structurally related outcomes in the Image, UI, and Architecture domains.

Achieving meaningful transfer requires careful consideration of semantic mapping. The symbolic weight of a shape can differ significantly across domains (e.g., a hexagon signifying 'efficiency' in UI data grids²¹ versus 'sacred harmony' in architecture¹⁹). The grammar or the prompt itself may need mechanisms to specify context or intended semantics alongside the geometric structure. Prompt archetypes like the "Cross-Domain Syntax Refactor" (Query) explicitly address this translation challenge, aiming to preserve both spatial logic and symbolic weight. This mapping process highlights both the potential power of a shared grammar for unified design thinking and the inherent complexities in bridging diverse domain conventions and meanings.

V. Recursion, Fractals, and Emergent Complexity

Recursion—the process of rules referring to themselves—is a powerful mechanism for generating complexity from simple beginnings. Integrating recursive principles into the polygonal grammar allows for the creation of intricate, nested, and potentially fractal structures.

A. Nested and Recursive Polygonal Structures

A polygonal grammar can employ recursive rules where a polygon symbol on the LHS of a rule is replaced by a structure that includes instances of the same symbol on the RHS. This iterative application allows for the generation of complex geometries that exhibit self-similarity across different scales.³⁸

Canonical examples of geometric structures generated through recursion include:

- **Koch Snowflake:** Starts with an equilateral triangle; each line segment is repeatedly replaced by a generator shape, creating an infinitely detailed perimeter.⁷
- **Sierpinski Triangle/Carpet:** Starts with a triangle or square; recursively removes smaller triangles or squares from the centers of the remaining shapes.
- **Nested Polygons:** Polygons contained within scaled versions of themselves.
- **L-System Generated Forms:** Branching structures simulating plants, spirals, and other organic or geometric patterns.³³

L-Systems provide a mature formalism for implementing such recursive generation.³³ Their string-rewriting mechanism naturally supports recursion. A rule like $A \rightarrow F[+A]F[-A]A$ defines how symbol A expands into forward movements (F), turns ($+$, $-$), branching ($[]$), and further instances of A , leading to exponential growth in complexity over generations.³³ L-systems can explicitly create polygonal geometry using commands like $\{$ (start polygon), $.$ (add vertex), and $\}$ (end polygon).³³

More advanced techniques like the **Recursive Fractal Modeling (RFM)** method aim to improve upon traditional approaches by offering enhanced control over symmetry, scaling, and self-similarity during recursive generation. RFM incorporates adaptive recursion (adjusting depth based on detail needed) and symmetry-preserving transformations.³⁴

B. Fractal Prompting: Implications and Challenges

Applying these recursive principles within prompts for generative AI models introduces the concept of "**fractal prompting**". This involves encoding recursive polygonal grammar structures directly into the instructions given to models like Stable Diffusion or Midjourney.

The potential effects of fractal prompting are twofold:

- **Enhanced Semantic Fidelity:** Recursive prompts could potentially capture complex hierarchical relationships or self-similar patterns that are difficult to describe purely textually, leading to more structured and intricate outputs.
- **Instability and Artifacts:** Current large generative models often struggle with precise spatial reasoning, complex geometric configurations, and maintaining consistency over intricate instructions.² A deeply nested or complex recursive geometric prompt might exceed the model's capacity for accurate interpretation. This could lead to the model ignoring parts of the recursive logic, generating chaotic or nonsensical outputs, producing geometric artifacts (distorted shapes, broken connections), or failing entirely to render the intended fractal structure [Query Lens 3].

The power of recursion, well-established in algorithmic generative art³³ and fractal geometry³¹, may not translate directly or reliably into the domain of *prompting* current AI systems. While L-systems provide a clear syntax for defining recursion algorithmically³³, expressing this same logic effectively within a prompt that a diffusion model can reliably parse and execute remains a significant challenge. The idea of reusable prompt macros (e.g., "tri-recursion:3x" from the query) is appealing for encapsulating complexity, but relies on the model's ability to understand and expand such macros correctly. Fractal prompting, therefore, represents a speculative frontier requiring substantial empirical investigation to determine its feasibility and limitations with existing and future generative architectures.

VI. Limitations, Failure Modes, and Critical Reflections

While a polygonal grammar system offers compelling possibilities, its practical implementation and conceptual underpinnings face significant limitations and potential pitfalls. A critical assessment requires examining challenges in grammar implementation, instabilities in generative models, symbolic conflicts, and inherent biases within the geometric formalism itself.

A. Grammar Implementation Challenges

1. **Ambiguity and Emergence:** A fundamental challenge inherited from shape grammar theory is the computational difficulty of recognizing **emergent** shapes—parts or configurations that arise through rule application but were not explicitly present in the initial structure or defined as primary parts.⁹ Creative design often relies on this phenomenon of "seeing as"⁸—reinterpreting existing forms in new ways.¹⁰ A rigidly defined polygonal grammar, especially one implemented within systems favoring fixed structures (like many CAD or potentially AI systems), might suppress this ambiguity and emergence, thereby limiting exploratory potential and producing overly constrained results.⁸ Enabling a promptable grammar to capture the fluidity of visual reinterpretation remains a difficult open problem.
2. **Computational Tractability:** Applying grammar rules, especially those involving complex geometric matching (like subgraph isomorphism for GGGs²⁶) or parametric shapes, can be computationally intensive.⁹ Recognizing all possible instances of a shape rule's LHS within a complex working shape, particularly if emergent configurations are allowed, can lead to combinatorial explosion. This may limit the grammar's usability in real-time or interactive generative systems.

B. Generative Model Instabilities and Artifacts

Current generative AI models, particularly diffusion models popular for image synthesis, exhibit weaknesses when dealing with precise geometric instructions:

1. **Geometric Inaccuracy:** Models like Midjourney and Stable Diffusion often struggle to render accurate geometric primitives, maintain straight lines, produce sharp corners, enforce consistent symmetry, or handle complex repeating patterns correctly.² Common artifacts include distorted or warped shapes, inconsistent angles, failure to close polygons, anatomically implausible structures in figures, and "stretched polygons" or other rendering glitches.² These inaccuracies could undermine the precision intended by a formal grammar.
2. **Prompt Misinterpretation:** Complex grammatical prompts, especially those involving deep nesting, recursion ("recursive hexatetragon" [Query Lens 5]), or subtle spatial relationships, may be poorly understood or partially ignored by the models. This can result in outputs that deviate significantly from the prompt's intent, feature nonsensical combinations of elements, or exhibit "hallucinated" structures not specified in the grammar.⁴¹ Users often employ **negative prompts** to explicitly forbid unwanted geometric features, artifacts, or common errors (e.g., "bad anatomy, bad proportions, geometry, mutation, distortion")⁴⁰, indicating the models' tendency towards such failures.
3. **Model Collapse:** Generative systems, especially if iteratively trained or prompted using synthetically generated data (potentially derived from the grammar itself), risk **model collapse**. This phenomenon involves the model's output becoming increasingly homogeneous, biased, error-prone, and degraded over time as it learns from its own potentially flawed creations.¹ Maintaining output quality and diversity when using a grammar for recursive or iterative generation might require careful curation and ensuring a sufficient proportion of real-world data in training.¹

C. Symbolic Conflicts and Over-Modularity

1. **Conflicting Affordances/Meanings:** The intended structural role of a polygon within the grammar might clash with its conventional symbolic meaning or perceived affordance in a specific domain [Query Lens 5]. For example, a triangle used structurally for stability might be misinterpreted visually as a warning symbol or an arrow indicating direction, leading to semantic confusion. The grammar needs mechanisms to resolve such ambiguities, perhaps through explicit context tagging or domain-specific rule sets.
2. **Over-Modularity and Sterility:** A strict adherence to modular rules and regular polygonal forms might lead to outputs perceived as mechanical, repetitive, or lacking the organic complexity and subtle imperfections often valued in design [Query Lens 5]. The grammar could potentially stifle creativity if it overly constrains the possibilities or makes it difficult to introduce intentional irregularities or deviations from the rules.

D. Inherent Biases: Euclidean, Symmetrical, and Cultural

The choice of a geometric formalism is not neutral and carries inherent biases:

1. **Euclidean Bias:** The default framework for most computational geometry and machine learning embeddings is **Euclidean space**. However, real-world data across various domains (language, vision, biology) often exhibits inherently non-Euclidean structures, such as hierarchies, multi-way relationships, symmetries, and non-uniform scaling.⁴² Representing these structures within Euclidean space can lead to significant distortion and require high-dimensional embeddings, resulting in computational inefficiency and potentially limiting representational power.⁴² For example, hierarchical concepts are often modeled orthogonally in Euclidean LLM embeddings, requiring dimensionality that scales poorly with complexity.⁴² Non-Euclidean geometries (like hyperbolic or spherical spaces) can often represent such structures more naturally and efficiently in lower dimensions.⁴² A polygonal grammar based purely on Euclidean primitives might inherit these limitations, struggling to effectively model complex relationships or non-uniform spatial logics.
2. **Symmetry and Regularity Bias:** A focus on regular polygons (triangle, square, hexagon) and rules emphasizing adjacency, alignment, and symmetry might inherently privilege orderly, modular, and symmetrical compositions. This could inadvertently suppress or devalue organic, irregular, asymmetric, or culturally diverse forms that do not easily conform to these geometric ideals. This reflects a broader critique of formal systems potentially overlooking the richness of non-standard or vernacular structures.⁸
3. **Cultural Universality Questioned:** The very foundation of the grammar in geometric primitives and rules raises questions about cultural bias. Is such a system inherently oriented towards Western design traditions, where Euclidean geometry, Platonic solids, and specific symbolic associations hold historical prominence?¹⁷ How effectively can it represent or adapt to diverse cultural symbolic systems¹⁴ or non-Western spatial logics and aesthetic principles? Meaning is culturally constructed¹⁴, and assuming universal interpretations of geometric forms is problematic.⁴⁴ A truly global grammar would need mechanisms for incorporating culturally specific symbols, rules, and meanings, moving beyond a potentially narrow geometric foundation.

The reliance on Euclidean geometry, therefore, is not merely a technical choice but one with potential representational and cultural implications. Overcoming these biases might necessitate exploring non-Euclidean geometries or more flexible topological approaches that can better accommodate the diverse structures found in real-world data and cultural expression.

The following table summarizes potential failure modes when applying the polygonal grammar.

Table 3: Failure Stack Matrix: Prompt Structures, Breakdown Types, and Domains

Prompt Structure/Completeness	Potential Breakdown Type	Domain: Image Generation	Domain: Interface Design	Domain: Architecture	Example Snippets
Basic Polygon Definition (e.g., Polygon(7))	Geometric Inaccuracy, Distortion	Misshapen heptagon, inconsistent angles/sides	Distorted button/icon shape	Inaccurate representation of a 7-sided room/element	2
Complex Adjacency/Tessellation (e.g., HexGrid)	Misalignment, Gaps/Overlaps, Pattern Incoherence	Broken/irregular tiling, inconsistent spacing	Uneven grid layout, overlapping UI elements	Poorly fitted modules, structural gaps	21
Strict Symmetry Rules (e.g., Symmetry(Reflect))	Broken Symmetry, Minor Asymmetries	Imperfect reflections, slightly skewed elements	UI elements not perfectly mirrored, alignment issues	Architectural features slightly off-center/asymmetric	2
Nested/Hierarchical Structures (e.g., A[C])	Loss of Detail, Structure Collapse, Incorrect Containment	Inner elements merge or disappear, scale issues	Nested menus fail to render, layout breaks	Internal spaces poorly defined or intersecting	32
Recursive/Fractal Prompts (e.g., Iterate(Rule, 5))	Model Instability, Chaos, Artifacts, Failure to Execute Logic	Nonsensical patterns, geometric noise, incomplete recursion	Unusable/crashing interface, infinite loops (conceptual)	Unbuildable/chaotic forms, structural incoherence	33
Conflicting Symbolic/Functional Roles	Semantic Ambiguity, Misinterpretation by User/Model	Visual confusion (e.g., stable shape used as warning)	Unclear affordance (e.g., triangle button meaning?)	Mismatch between form and perceived function	16
High Density/Complexity	Over-Cluttering, Performance Issues, Model Collapse Risk	Visually noisy image, loss of focus	Slow/unresponsive UI, information overload	Impractical density, construction complexity	1
Non-Euclidean Concepts (if attempted)	Model Incapacity, Euclidean Projection Artifacts	Failure to render curvature/topology, distortions	Layout fails on non-planar surfaces (conceptual)	Inaccurate representation of curved/topological space	42

VII. Conclusion: Prospects for Polygonal Grammar in Generative Futures

A. Synthesis of Findings

This investigation into a formal Polygonal Grammar System reveals a concept with significant theoretical promise, yet one facing considerable practical and conceptual hurdles. The potential to establish a structured, symbolic language for controlling generative AI across diverse design domains—image generation, interface design, and

architecture—is compelling. Building on the legacy of shape grammars³ and leveraging the inherent semiotic potential of geometric forms¹¹, such a grammar could offer enhanced predictability, control, and cross-domain transferability in prompt-based workflows. Specific geometric properties, like the efficiency of hexagonal tessellations for UI grids²¹, highlight potential concrete advantages.

However, the analysis also underscores critical challenges. The computational difficulty of implementing grammars that capture **emergence and ambiguity**—key aspects of creative design recognized in shape grammar theory⁸—persists. Current generative models exhibit significant limitations in accurately rendering complex geometry and interpreting intricate, recursive grammatical instructions², potentially leading to artifacts, instability, or failure to adhere to the prompt's logic. Furthermore, the system must grapple with **semantic ambiguity** (conflicting meanings of shapes across contexts¹⁶) and the risk of **cultural bias** inherent in privileging specific geometric forms and Euclidean space.¹⁴ The potential for over-modularity leading to sterile outputs also requires consideration [Query Lens 5]. The possibility of learning grammars via inverse procedural modeling offers a promising avenue²⁶, but is itself a complex research area.

B. Future Research Directions

Realizing the potential of polygonal grammars requires focused research efforts:

1. **Tooling and Empirical Testing:** Development of prototype tools, prompt libraries, or visual programming interfaces based on the proposed syntax (Table 2). Rigorous empirical testing is needed to evaluate the feasibility and effectiveness of using such grammars with current state-of-the-art generative models across the target domains.
2. **Non-Euclidean and Topological Extensions:** Investigating the integration of non-Euclidean geometries (hyperbolic, spherical)⁴² or purely topological relationship descriptors to overcome the limitations and biases of a strictly Euclidean framework. This could enhance representational power and efficiency.
3. **Cultural Adaptation and Semantics:** Researching methods to make the grammar more culturally adaptable. This could involve incorporating parameters for cultural context, allowing user-defined symbolic libraries, or developing mechanisms for handling polysemy and semantic nuance explicitly within the grammar.¹⁴
4. **Grammar Learning and Inverse Modeling:** Advancing techniques for automatically learning effective polygonal grammar rules from existing design corpora (images, UI layouts, architectural plans).²⁶ This could lead to more powerful and contextually relevant grammars.
5. **Improving AI Geometric Capabilities:** Continued research within the AI community is needed to enhance the fundamental capabilities of generative models in understanding and executing precise geometric and spatial instructions, including complex recursion and symmetry constraints.
6. **Addressing Ambiguity and Emergence:** Exploring computational techniques, perhaps drawing from perception psychology or advanced shape analysis, to allow grammar implementations to recognize and leverage emergent structures, moving beyond fixed part definitions.⁹

C. Concluding Thoughts

The exploration of polygonal grammar systems sits at the intersection of geometry, computation, semiotics, and design practice. It represents a move towards more structured, symbolic, and potentially more controllable methods for interacting with generative AI. While the vision of a universal geometric language seamlessly bridging design domains is alluring, its realization is contingent on overcoming significant technical challenges in both grammar implementation and AI model capabilities, as well as critically addressing the inherent biases embedded

within any formal system.

Ultimately, the value of such a grammar may lie not only in its potential for precise generation but also in its capacity to externalize and formalize design thinking, forcing a clearer articulation of structural and semantic intent. Whether this formalization ultimately enhances or constrains creativity remains an open question, likely dependent on the flexibility of the grammar and the skill of the designer wielding it. The pursuit of geometrically-informed generative methods promises to be a rich area for future research, potentially reshaping how designers conceive, communicate, and create with intelligent machines.

Works cited

1. Could we see the collapse of generative AI? - Inria, accessed May 12, 2025, <https://www.inria.fr/en/collapse-ia-generatives>
2. Characterizing Photorealism and Artifacts in Diffusion Model-Generated Images - Matt Groh, accessed May 12, 2025, <https://mattgroh.com/pdfs/2502.11989v1.pdf>
3. Shape grammar - Wikipedia, accessed May 12, 2025, https://en.wikipedia.org/wiki/Shape_grammar
4. Shape Grammar - Radical Art, accessed May 12, 2025, <http://radicalart.info/AlgorithmicArt/grammar/ShapeGrammar/index.html>
5. Using Shape Grammar With PCG in Unreal Engine - Epic Games Developers, accessed May 12, 2025, <https://dev.epicgames.com/documentation/en-us/unreal-engine/using-shape-grammar-with-pcg-in-unreal-engine>
6. Generative grammars as a form of procedural content generation - The Shaggy Dev, accessed May 12, 2025, <https://shaggydev.com/2022/03/16/generative-grammars/>
7. Explaining a Jewelry Design Process: Using a generative Design Method Called Shape Grammars and Producing the Product with 3D Printing - Scientific Research Publishing, accessed May 12, 2025, <https://www.scirp.org/journal/paperinformation?paperid=129887>
8. SHAPE GRAMMARS: SEVEN QUESTIONS AND THEIR SHORT ANSWERS 1. A. M. Turing, "On Computable Numbers, with an Application to the En - MIT Press Direct, accessed May 12, 2025, https://direct.mit.edu/books/oa-monograph/chapter-pdf/2232973/c002600_9780262370653.pdf
9. Shapes, structures and shape grammar implementation - Open Research Online, accessed May 12, 2025, <https://oro.open.ac.uk/59348/8/CAD19-submitted.pdf>
10. Craft, Performance, and Grammars | Computational Making, accessed May 12, 2025, <http://compmaking.scripts.mit.edu/index/wp-content/uploads/2017/02/KNIGHT-paper.pdf>
11. SHAPES: SEMIOTICS + SYMBOLISM - BRAD DAILEY, accessed May 12, 2025, <https://www.braddailey.com/shapes-semiotics-and-symbolism>
12. (PDF) Semiotic Anthropology - ResearchGate, accessed May 12, 2025, https://www.researchgate.net/publication/228156081_Semiotic_Anthropology
13. The Semiotic Structure of Geometry Diagrams: How Textbook Diagrams Convey

- Meaning, accessed May 12, 2025,
https://www.nctm.org/Publications/journal-for-research-in-mathematics-education/2015/Vol46/Issue2/The-Semiotic-Structure-of-Geometry-Diagrams_-How-Textbook-Diagrams-Convey-Meaning/
14. Symbolic and Interpretive Anthropologies, accessed May 12, 2025,
<https://anthropology.ua.edu/theory/symbolic-and-interpretive-anthropologies/>
 15. The theory of cultural logic: How individuals combine social intelligence with semiotics to create and maintain cultural meaning - Max Planck Institute for Psycholinguistics, accessed May 12, 2025,
<https://www.mpi.nl/publications/item94904/theory-cultural-logic-how-individuals-combine-social-intelligence-semiotics>
 16. A Guide to Psychology of Shapes in Design - Pixcap, accessed May 12, 2025,
<https://pixcap.com/blog/psychology-of-shapes>
 17. Sacred Geometry: The Universal Patterns in Art and Architecture ..., accessed May 12, 2025,
<https://homeearthaven.com/blogs/news/sacred-geometry-the-universal-patterns-in-art-and-architecture>
 18. Sacred geometry | EBSCO Research Starters, accessed May 12, 2025,
<https://www.ebsco.com/research-starters/religion-and-philosophy/sacred-geometry>
 19. Why In Design: The Long History of Hexagon Shapes in Our Home & Architecture, accessed May 12, 2025,
<https://nestnestnest.blogspot.com/2016/09/why-in-design-long-history-of-hexagon.html>
 20. The hidden meaning behind the hexagon - Flower of Sound, accessed May 12, 2025,
<https://www.flowerofsound.com/the-hidden-meaning-behind-the-hexagon/>
 21. H3 Hexagonal Grid - Kontur Inc., accessed May 12, 2025,
<https://www.kontur.io/blog/why-we-use-h3/>
 22. Fishnets and Honeycomb: Square vs. Hexagonal Spatial Grids | Matt Strimas-Mackey, accessed May 12, 2025,
<https://strimas.com/post/hexagonal-grids/>
 23. Star polygons in art and culture - Wikipedia, accessed May 12, 2025,
https://en.wikipedia.org/wiki/Star_polygons_in_art_and_culture
 24. arXiv:2301.01471v1 [cs.GR] 4 Jan 2023, accessed May 12, 2025,
<https://arxiv.org/pdf/2301.01471>
 25. Creating Star Patterns on the Sphere 1 Introduction - ResearchGate, accessed May 12, 2025,
https://www.researchgate.net/profile/Peter-Cromwell/publication/374443100_Creating_Star_Patterns_on_the_Sphere/links/651da2a9fc5c2a0c3bb60624/Creating-Star-Patterns-on-the-Sphere.pdf
 26. Learning Geometric Graph Grammars - CS@Purdue, accessed May 12, 2025,
https://www.cs.purdue.edu/cgvlab/www/resources/papers/Fiser-2016-Learning_Geometric_Graph_Grammars.pdf
 27. Graph Grammars Based on Node Rewriting: An Introduction to NLC Graph

- Grammars., accessed May 12, 2025,
https://www.researchgate.net/publication/220713290_Graph_Grammars_Based_on_Node_Rewriting_An_Introduction_to_NLC_Graph_Grammars
28. [2301.01471] Freeform Islamic Geometric Patterns - ar5iv - arXiv, accessed May 12, 2025, <https://ar5iv.labs.arxiv.org/html/2301.01471>
 29. Composition Basics (3): Centre Composition, Symmetrical Composition - SNAPSHOT, accessed May 12, 2025,
<https://snapshot.canon-asia.com/article/eng/part-3-composition-basics-center-composition-and-symmetrical-composition>
 30. Composition Techniques & Design Principles for Graphic Designers - YesImADesigner.com, accessed May 12, 2025,
<https://yesimadesigner.com/composition-techniques/>
 31. Fractal - Wikipedia, accessed May 12, 2025, <https://en.wikipedia.org/wiki/Fractal>
 32. Build responsive navigation | Views - Android Developers, accessed May 12, 2025,
<https://developer.android.com/develop/ui/views/layout/build-responsive-navigation>
 33. L-System - SideFX, accessed May 12, 2025,
<https://www.sidefx.com/docs/houdini/nodes/sop/lssystem.html>
 34. Mathematical Modeling and Recursive Algorithms for Constructing ..., accessed May 12, 2025, <https://www.mdpi.com/2227-7390/13/4/646>
 35. THE DESIGN AND EVALUATION OF GRAPHICAL RADIAL MENUS - University of Galway Research Repository, accessed May 12, 2025,
https://researchrepository.universityofgalway.ie/bitstream/handle/10379/2672/Samp_PhD_RadialMenus.pdf?isAllowed=y&sequence=1
 36. Pie menu - Wikipedia, accessed May 12, 2025,
https://en.wikipedia.org/wiki/Pie_menu
 37. Browse thousands of Nested Navigation images for design inspiration | Dribbble, accessed May 12, 2025, <https://dribbble.com/search/nested-navigation>
 38. Reading #1: What is Generative Art? - NYU Web Publishing, accessed May 12, 2025,
<https://wp.nyu.edu/nyushanghai-fatimakazim/2023/09/12/reading-1-what-is-generative-art/>
 39. Interesting artifacts/glitches in renders - Blender Artists Community, accessed May 12, 2025,
<https://blenderartists.org/t/interesting-artifacts-glitches-in-renders/1513845>
 40. 180+ Best Stable Diffusion Negative Prompts with Examples - Aiarty Image Enhancer, accessed May 12, 2025,
<https://www.aiarty.com/stable-diffusion-prompts/stable-diffusion-negative-prompt.htm>
 41. Generative AI Glitch Art: Looking for meaning in all the wrong places - YouTube, accessed May 12, 2025, <https://www.youtube.com/watch?v=r-V-qgLLJF8>
 42. Foundation Models Should Embrace Non-Euclidean Geometries - arXiv, accessed May 12, 2025, <https://arxiv.org/html/2504.08896v1>
 43. Does a specific someone have a bias against Mathematical Structuralism? [closed], accessed May 12, 2025,

<https://philosophy.meta.stackexchange.com/questions/6390/does-a-specific-someone-have-a-bias-against-mathematical-structuralism>

44. 16 Cross-cultural differences as meaning systems - Indigenous Psychology, accessed May 12, 2025,

[https://indigenoupsych.org/Discussion/forum/PDF/cross-cultural%20meaning\(4\).pdf](https://indigenoupsych.org/Discussion/forum/PDF/cross-cultural%20meaning(4).pdf)